

Análisis y Diseño con Patrones

PAC 1: Principios de Diseño y Patrones de Análisis

Pregunta 1 (20%)

Imaginemos una empresa de desarrollo de software especializada en software de ingeniería. El software desarrollado debe poder trabajar con varias unidades de medida a la vez (ya sean del sistema métrico o de otro), pero actualmente las unidades no se guardan explícitamente. Este hecho ha ocasionado problemas en un par de ocasiones, ya que se han encontrado errores porque una función pasaba un valor a otra y cada función asumía que el valor estaba en una unidad de medida diferente (por error)¹.

Nos proponemos desarrollar un nuevo módulo para el software de esta empresa. Antes de hacer el diagrama estático de análisis, sin embargo, revisemos los patrones de análisis buscando uno que pueda ayudar a resolver el problema comentado. Indica qué patrón de análisis habría que tener especialmente en cuenta y justifica brevemente el por qué.

Solución

Este es un caso muy claro en que el patrón Cantidad nos puede ayudar. Tal como expresa el patrón, en este caso nos encontramos con que la representación de una cantidad simplemente con un número nos ha dado problemas y, además, no es viable si queremos representar cantidades utilizando unidades diferentes. Por tanto, la descripción del problema que el patrón resuelve encaja perfectamente con el problema que necesitamos resolver en este caso.

Además, la solución propuesta por el patrón es aplicable al problema. Se trata de modelar la medida como un tipo de datos más complejo formado por un valor y una unidad.

¹ https://es.wikipedia.org/wiki/Mars_Climate_Orbiter (ver “La Mars Climate Orbiter se destruyó debido a un error de navegación...”)

Pregunta 2 (20%)

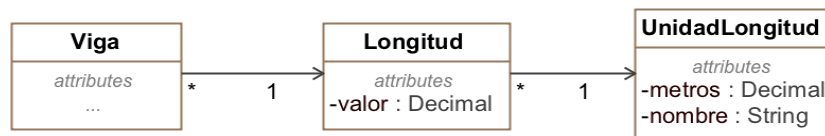
En el mismo contexto de la pregunta 1, queremos desarrollar un módulo que permita realizar cálculos basados en longitudes. Concretamente, necesitaremos:

- Representar longitudes en varias unidades de medida, por ejemplo, 12,3 cm o 4,2 pulgadas
- Permitir a los usuarios finales crear nuevas unidades de medida cuando las necesiten. Así, las unidades de medida utilizables se deben poder definir de forma dinámica.
- Que el sistema conozca cómo convertir de cualquier unidad de longitud a cualquier otra.

Propón un diagrama estático de análisis donde se aplique el patrón de análisis identificado en la pregunta 1 para los requisitos descritos. Muestra, concretamente, cómo se modelaría una supuesta clase *Viga*, que en la versión actual del software dispone de un atributo *longitud* de tipo Decimal que representa la longitud de la viga en centímetros. Muestra también cómo se guarda la ratio de conversión entre unidades de longitud diferentes.

Nota: Si es necesario, haz alguna variación sobre el patrón tal como se explica en los materiales para poder satisfacer todos los requisitos enumerados.

Solución



Tal como indica el patrón, se ha modelado la longitud como un tipo de datos adicional. En este caso, sin embargo, en lugar de indicar la unidad como un String, se ha modelado una clase para las unidades de longitud. Esto permite modelar también el ratio de conversión entre unidades. Para ello, se ha tomado una unidad base (se ha elegido el metro) y se ha modelado un atributo que nos permite guardar cualquier ratio de conversión hay entre cada unidad y la unidad base.

Pregunta 3 (30%)

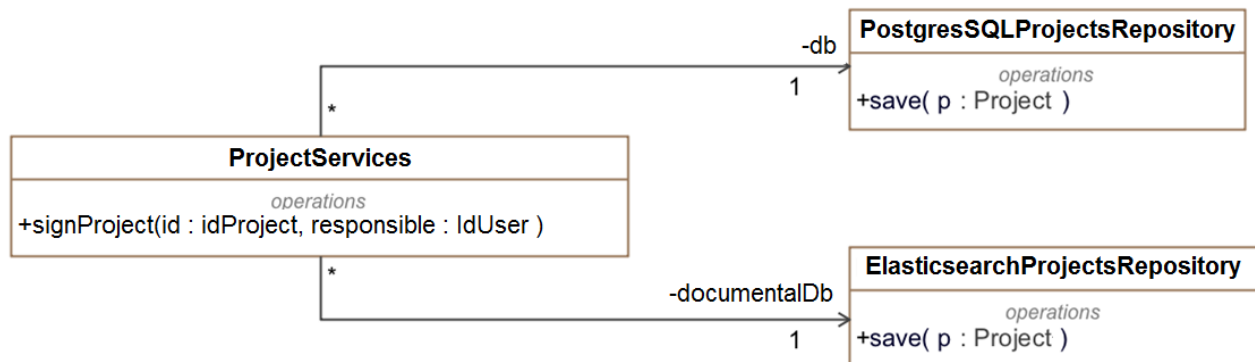
Uno de los principios de diseño que debe cumplir un sistema para obtener un nivel alto de calidad es el principio de Inversión de dependencias, es decir, que los módulos y clases de alto nivel no deberían depender los de bajo nivel sino de una abstracción. Investiga y conoce a fondo este principio de diseño (puedes mirar la bibliografía recomendada y encontrar muchos recursos por internet, aparte de los propios módulos de la asignatura).

Una vez entendido el principio, situémonos en un caso imaginario en el que un determinado sistema de información, por razones tecnológicas, debe guardar los datos de forma persistente en dos almacenes de datos: una base de datos relacional (PostgreSQL) y una base de datos documental (Elasticsearch). Supongamos que se trata de un sistema de gestión de proyectos de ingeniería en el que un responsable técnico, en un determinado momento, revisa un proyecto y da el visto bueno. Supongamos también que tenemos una clase de alto nivel de abstracción (*ProjectServices*) que tiene una operación *signProject* para dar el visto bueno al proyecto por parte del responsable.

Un posible diseño consistiría en controlar la persistencia desde el código de dominio. Simplificando mucho el problema, el diseño de *signProject* expresado en pseudocódigo, podría ser:

```
public void signProject(IdProject id, IdUser responsable) {
    Project p = db.getProject(id);
    // validar las reglas de negocio que indican si se puede aceptar o no el proyecto
    ...
    // realizar las modificaciones de negocio al proyecto cuando se valida, como guardar quién firma
    ...
    db.save(p);
    documentalDb.save(p);
}
```

En este ejemplo, *db* y *documentalDb* son dos objetos que dan acceso a las funciones de almacenamiento en base de datos relacional y documental respectivamente. En un diagrama de clases, podríamos representarlo como:



- Este ejemplo constituye o puede constituir una violación del principio de inversión de dependencias. ¿Por qué?
- ¿Qué consecuencias tendría esta violación?

Solución

- Por definición del principio de diseño, este ejemplo constituye una violación del principio de inversión de dependencias si *PostgreSQLProjectsRepository* o *ElasticsearchProjectsRepository* son clases de más bajo nivel que *ProjectServices*. A juzgar por los nombres de las clases, este parece ser el caso.
- La principal consecuencia negativa es que la clase de más alto nivel de abstracción (*ProjectServices*), que es la que contiene la lógica de negocio, está acoplada a detalles técnicos de implementación y, por tanto, puede verse afectada por cambios de bajo nivel (en este caso, de tecnologías de almacenamiento), que no le deberían afectar.

Pregunta 4 (30%)

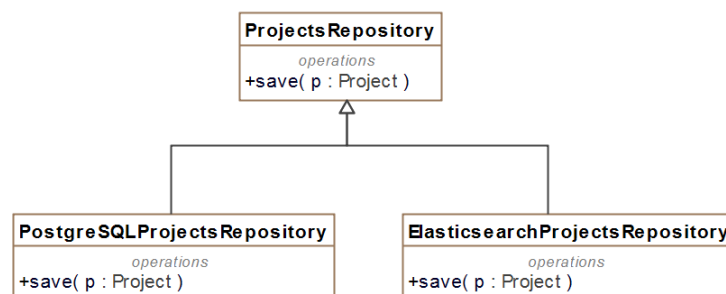
Propón cómo se podría diseñar el sistema anterior para que se satisfaga el principio de inversión de dependencias.

Usa diagramas estáticos de análisis, diagramas de secuencia y / o pseudocódigo de operaciones según creas más relevante para explicar la solución propuesta. Razona, también, como la solución supera las consecuencias negativas del apartado *b* de la pregunta anterior.

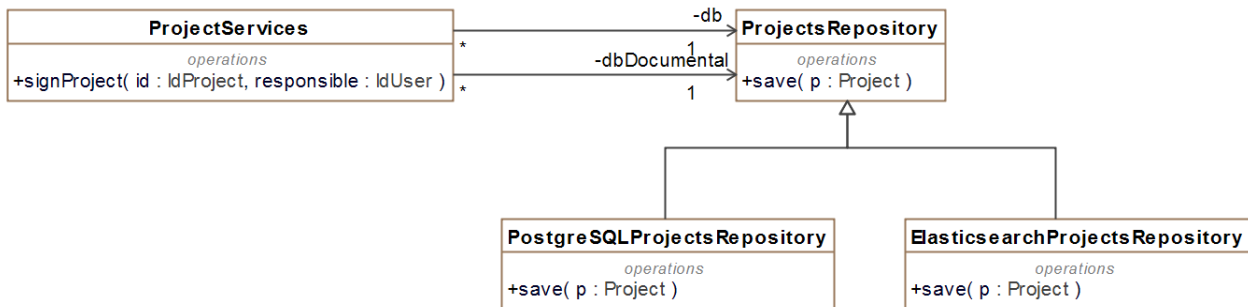
Solución

Según el principio de inversión de dependencias, lo que necesitamos es crear una abstracción de alto nivel que las clases más concretas deberán implementar. Esta abstracción pertenecerá a *ProjectServices*, en el sentido de que será de su mismo alto nivel de abstracción y formará parte del mismo módulo que ésta.

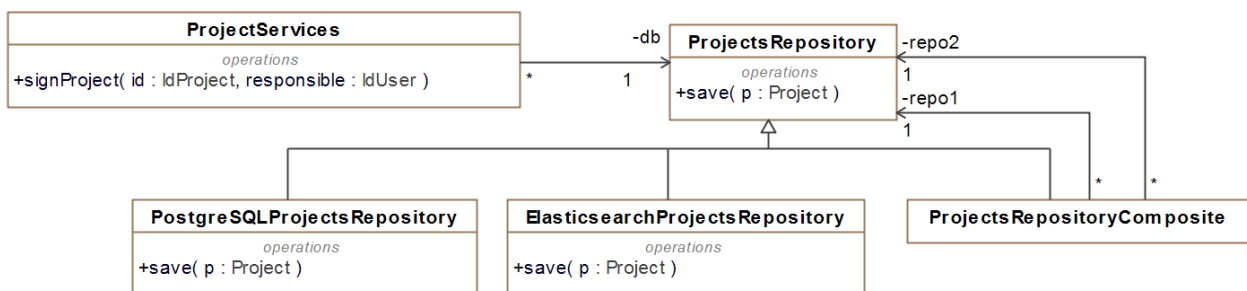
La abstracción común a ambos mecanismos de persistencia podría ser la misma:



Entonces lo que hay que resolver es cómo hacer que la clase *ProjectServices* no esté acoplada a las clases concretas sino a la abstracción:



Llegados a este punto, podemos detectar que ya sólo dependemos de abstracciones... pero que el hecho de que necesitamos dos instancias de *ProjectsRepository* en lugar de sólo una es, también, un detalle tecnológico de bajo nivel. Podríamos, pues, crear una nueva abstracción que represente los dos mecanismos de almacenamiento:



De esta manera, *ProjectServices* sólo tendría un repositorio de proyectos. En el caso del repositorio compuesto, la clase concreta, de bajo nivel, sería la que compondría el almacenamiento de proyectos como dos invocaciones a *save* de dos repositorios diferentes.

En *ProjectServices*:

```

public void signProjec (IdProject id, IdUser responsable){
    Project p = db.getProject(id);
    // Validar las reglas de negocio que indican si se puede aceptar o no el proyecto,
    ...
    // hacer las modificaciones de negocio al proyecto cuando se valida, como guardar quien lo firma
    ...
    db.save(p);
}
  
```

La implementación de *ProjectsRepositoryComposite* sería:

```
public void save (Project p){  
    repo1.save(p);  
    repo2.save(p);  
}
```

Esta solución protege la clase *ProjectServices* de cambios tecnológicos. Un cambio tecnológico de almacenamiento de datos sólo la podría afectar si afectara la abstracción que hemos creado: *ProjectsRepository*. Como la abstracción es totalmente agnóstica de la tecnología, esta probabilidad de cambio se ha minimizado.